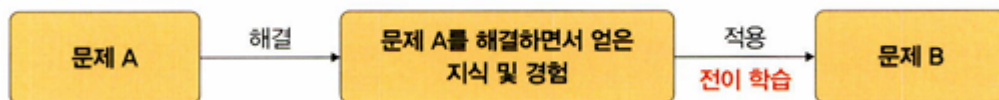


5장_합성곱 신경망 1_Part 2(5.3~)

5.3. 전이 학습 (transfer learning)

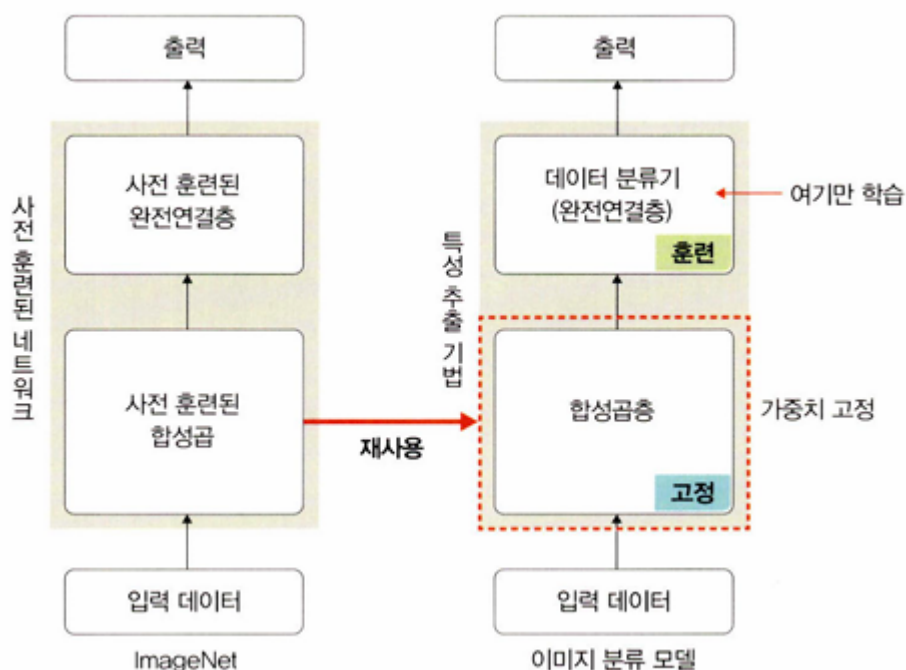
▼ 그림 5-29 전이 학습



이미지넷(ImageNet)처럼 사전 훈련된 모델의 가중치를 가져와 해결하려는 과제에 맞게 보정해서 사용하는 것

- 사전 훈련된 모델(네트워크): 아주 큰 데이터셋으로 훈련된 모델
- ImageNet: 영상 인식 기술의 성능을 평가하는 주된 이미지 데이터셋. 클래스 2만 개 이상, 이미지 총 1419만 7122장으로 구성됨.
- 합성곱 신경망 기반 딥러닝 모델을 제대로 훈련시키려면 많은 양의 데이터가 필요한데, 큰 데이터셋을 확보하지 않고도 원하는 과제를 해결할 수 있도록 해 주는 방법
- 전이 학습 방법: 특성 추출, 미세 조정 기법

5.3.1. 특성 추출(feature extractor) 기법 — 실습



ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후 마지막에 **완전연결층 부분만 새로 만들어**, 학습할 때는 **마지막 완전연결층**(이미지의 카테고리를 결정하는 부분)만 학습하고 나머지 계층들은 학습되지 않도록 함

- 구성
 - 합성곱층: 합성곱층 + 풀링층
 - 데이터 분류기(완전연결층): 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스 분류
- 사용 가능한 이미지 분류 모델
: Xception, Inception V3, ResNet50, VGG16, VGG19, MobileNet

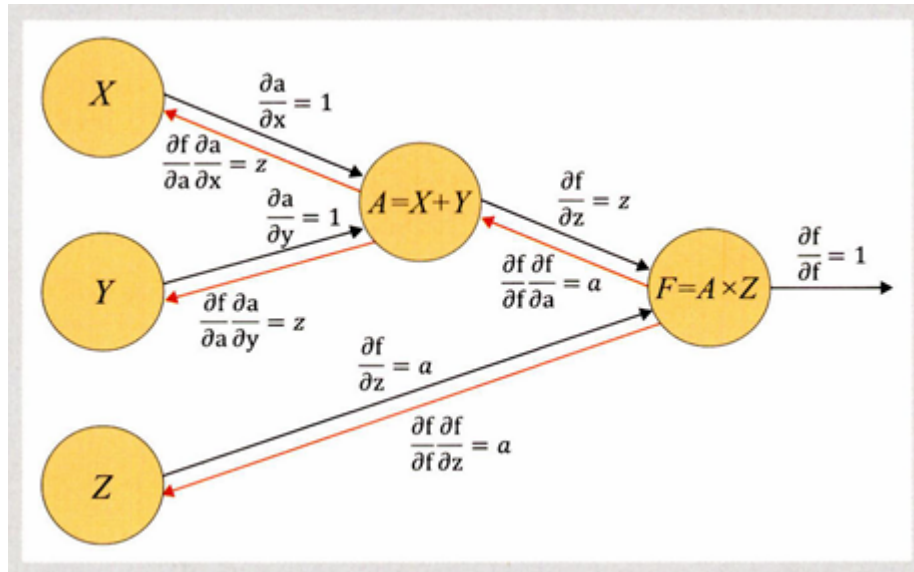
▼ 사전 훈련된 모델

파이토치에서 아래 코드처럼 무작위의 가중치로 모델 구성 가능

```
import torchvision.models as models
resnet18 = models.resnet18()
alexnet = models.alexnet()
vgg16 = models.vgg16()
squeezenet = models.squeezenet1_0()
densenet = models.densenet161()
inception = models.inception_v3()
googlenet = models.googlenet()
shufflenet = models.shufflenet_v2_x1_0()
mobilenet_v2 = models.mobilenet_v2()
mobilenet_v3_large = models.mobilenet_v3_large()
mobilenet_v3_small = models.mobilenet_v3_small()
resnext50_32x4d = models.resnext50_32x4d()
wide_resnet50_2 = models.wide_resnet50_2()
mnasnet = models.mnasnet1_0()
```

사전 학습된 모델의 가중치 값을 사용하고 싶으면 파라미터로 `pretrained=True` 입력

▼ 계산 그래프 (computational graph)



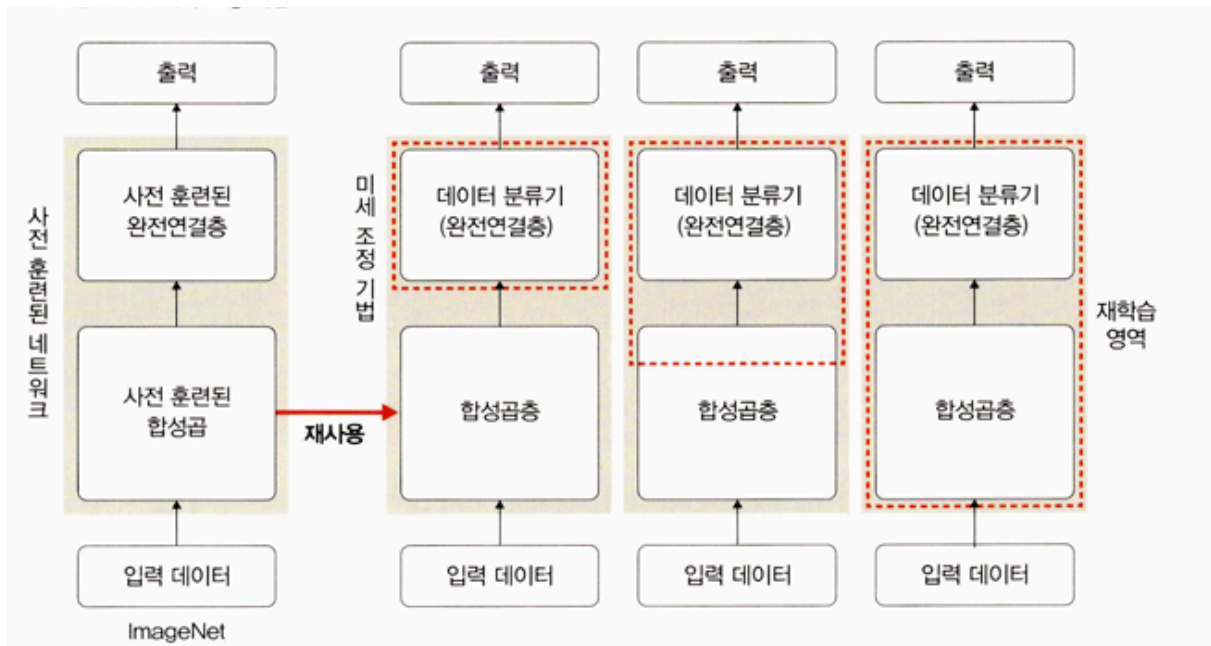
- 계산 그래프: 계산 과정을 **그래프**로 나타낸 것
- 계산 그래프를 사용하는 이유
 - 국소적 계산 가능
 - 역전파를 통한 미분 계산이 편리함: 주황색 선에서 연쇄 법칙(chain rule)을 이용해 빠르고 간편하게 미분 계산하고 있음

5.3.2. 미세 조정 기법 (fine-tuning)

사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식

⚠️ 특성 추출: 목표 특성을 잘 추출했다는 전제하에 좋은 성능을 낼 수 있지만, 특성이 잘못 추출되었다면 미세 조정 기법으로 새로운 이미지 데이터를 사용해 네트워크의 가중치를 업데이트해 특성을 다시 추출할 수 있음

→ 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습시키는 것



- 사전 훈련된 **네트워크**를 **미세 조정**해 분석하려는 데이터셋에 잘 맞도록 모델의 파라미터를 조정하는 기법
- 많은 연산량 필요 → gpu 사용 권장
- 미세 조정 기법 전략
 - 데이터셋이 크고 사전 훈련된 모델과 유사성이 작음: 모델 전체 재학습
 - 데이터셋이 크고 사전 훈련된 모델과 유사성이 큼: 합성곱층 뒷부분, 데이터 분류기 학습
 - 데이터셋이 작고 사전 훈련된 모델과 유사성이 작음: 합성곱층 일부분, 데이터 분류기 학습
 - 데이터 적으면 일부 계층에 미세 조정 기법을 적용해도 효과가 없을 수 있음
 - 합성곱층 중 어디까지 새로 학습시킬지 적당히 설정해 주어야 함
 - 데이터셋이 작고 사전 훈련된 모델과 유사성이 큼: 데이터 분류기만 학습
 - 데이터셋이 작아 많은 계층에 미세 조정 기법 적용하면 과적합 위험

5.4. 설명 가능한 CNN (explainable CNN)

딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술

✅ CNN 처리 과정(입력 이미지에서 특성이 어떻게 추출되고 학습되는지)을 시각화함으로써 CNN으로 얻은 결과의 신뢰성을 높임

- CNN 시각화 방법: 필터에 대한 시각화, 특성 맵에 대한 시각화

5.4.1. 특성 맵(feature map) 시각화 — 실습

특성 맵 (혹은 활성화 맵)

- 입력 이미지 또는 필터를 입력에 적용한 결과
- 특성 맵에서 입력 특성을 감지하는 방법을 이해할 수 있도록 돕는 것

PIL(Python Image Library) : 다양한 이미지 파일 형식에 대해 이미지 분석 및 처리를 도와주는 이미지 프로세싱 라이브러리

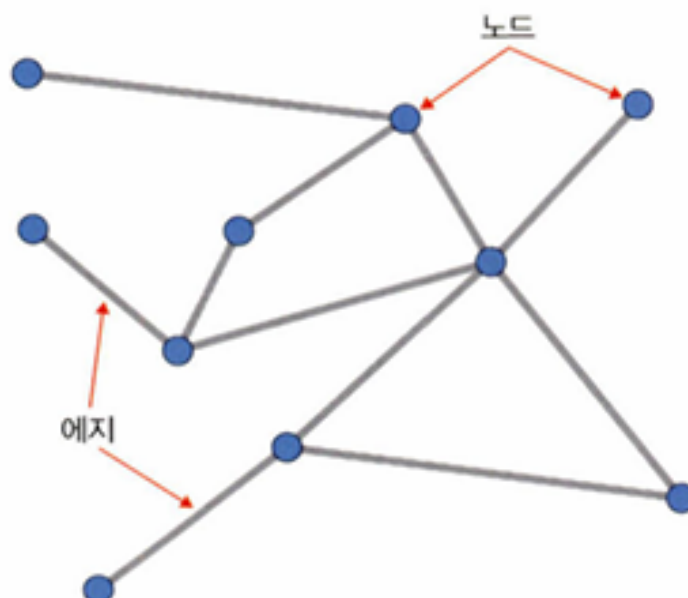
5.5. 그래프 합성곱 네트워크 (graph convolutional network)

그래프 데이터를 위한 신경망

5.5.1. 그래프란?

방향성이 있거나(directed) 없는(undirected) 엣지로 연결된 노드(nodes=vertices)의 집합

▼ 그림 5-50 그래프



- 그래프 구성 요소

- 노드(node, vertex): 원소
- 엣지(edge): 두 노드 연결한 선, 결합 방법

5.5.2. 그래프 신경망 (Graph Neural Network, GNN)

그래프 구조에서 사용하는 신경망

그래프 데이터의 표현

1. 인접 행렬(adjacency matrix)

- 노드 n 개를 $n \times n$ 행렬 (matrix) 로 표현
- 인접 행렬의 값: 'A_{ij} 는 i와 j의 관련성 여부' 를 만족하는 값으로 채움

2. 특성 행렬(feature matrix)

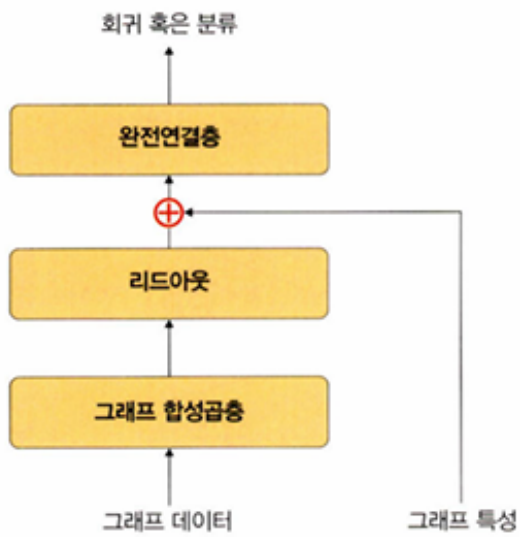


- 인접 행렬만으로는 특성 파악이 어려움 → 다누이 행렬 적용
- 각 입력 데이터에서 이용할 특성 선택
- 특성 행렬의 각 행: 선택된 특성에 대해 각 노드가 갖는 값
(ex. 첫 행 = 첫 번째 노드의 특성 값)
- 특성 행렬 과정을 통해 그래프 특성(graph feature) 추출

5.5.3. 그래프 합성곱 네트워크(GCN)

이미지에 대한 합성곱 → 그래프 데이터에 대한 합성곱

▼ 그림 5-52 그래프 합성곱 네트워크



- 리드아웃(readout): 특성 행렬을 하나의 벡터로 변환하는 함수
 - 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터 생성
- **그래프 합성곱층(graph convolutional layer)**
 - 그래프 합성곱층을 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환되므로, 딥러닝 알고리즘 적용 가능
(기존 그래프 형태 데이터로는 딥러닝 알고리즘 적용 불가능)

✓ GCN의 활용

- SNS 관계 네트워크
- 학술 연구의 인용 네트워크
- 3D Mesh